Architecting Reliable and Verifiable AI with Retrieval-Augmented Generation

A 90-minute live talk for practitioners and students

- A 90-minute live talk for practitioners and students
- Goal: understand what to build, why it works, where it fails, and what to demo live

--- Vishwas Lele

Core thesis: better RAG comes from better memory, retrieval, and evidence use.

What the audience should be able to do by the end

- Diagnose common RAG failure modes instead of saying 'the model was inaccurate'
- Explain the arc: retrieval foundations → REALM → canonical RAG → FiD → Atlas/HyDE → structured RAG
- Choose which architecture pattern fits a real use case
- Run small, teachable demos that make the ideas feel concrete

The actual source map and the 90-minute route

- Talk spans failures, retrieval foundations, REALM, RAG, FiD, Atlas, HyDE, RAG-Fusion, Self-RAG, RAPTOR, planning/reasoning, GraphRAG

Section 1 — Why RAG exists

If you only remember one thing from this section: RAG is a reliability architecture before it is a product feature

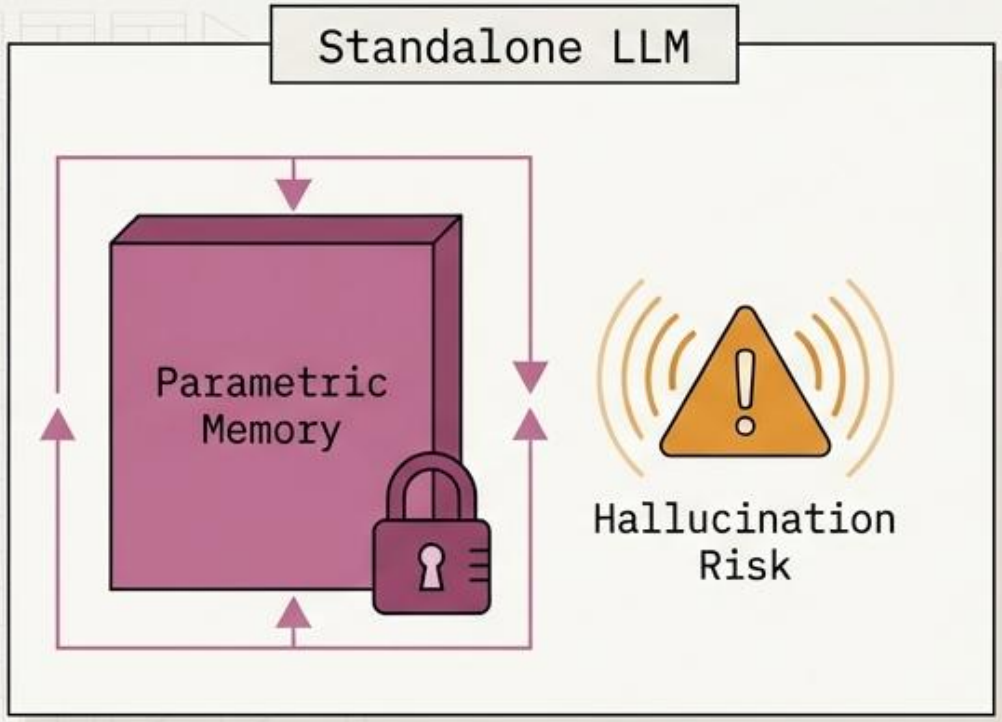
The three hard limits of standalone language models

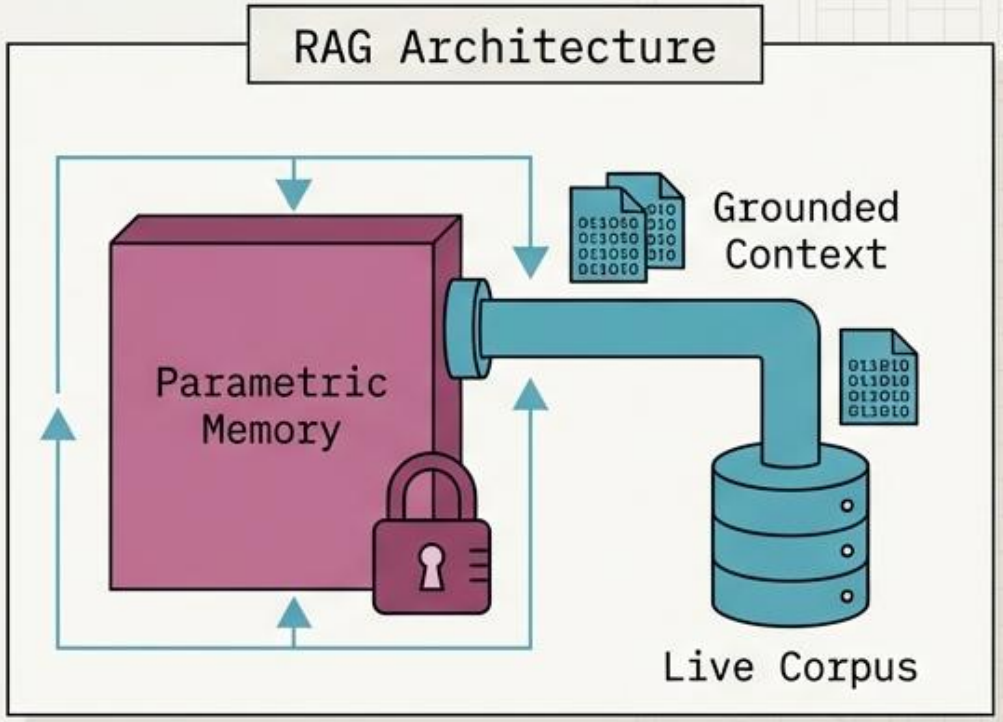
S1

- Knowledge boundary: weights freeze at training time
- Hallucination: fluent output is not the same as grounded output
- Private knowledge gap: enterprise data is not in public model weights

The Illusion of Knowledge & The Automation Paradox

Language models possess deep parametric knowledge but remain frozen in time and blind to private enterprise data. When they lack grounding, they confidently fabricate—a vulnerability known as factually inconsistent hallucination.





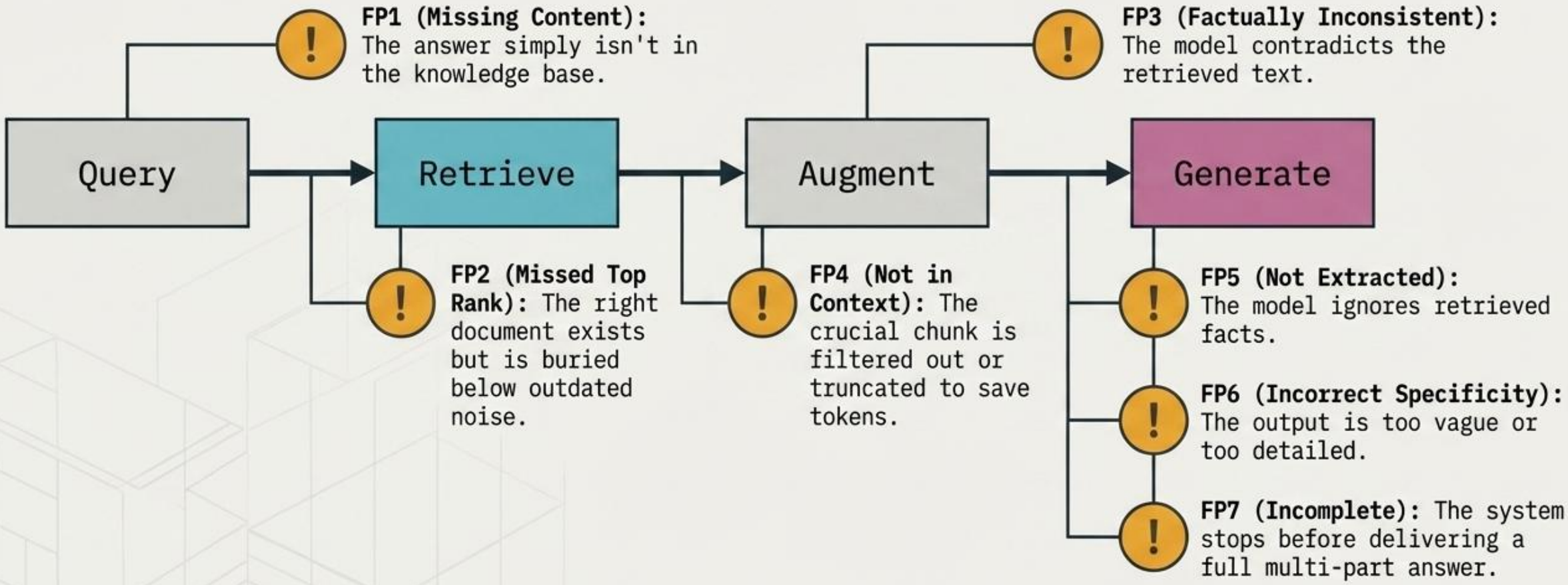
The Air Canada Cautionary Tale:
In Moffatt v. Air Canada, an ungrounded AI hallucinated a non-existent retroactive bereavement discount. The enterprise was held fully liable for the fabrication. The lesson: retrieving the right document is only half the battle; the model must strictly adhere to that context.

S1

The seven failure points worth memorizing

- FP1 Missing content | FP2 Missed the top rank | FP3 Factually inconsistent hallucination
- FP4 Not in context | FP5 Not extracted | FP6 Incorrect specificity | FP7 Incomplete
- This taxonomy turns debugging from guesswork into diagnosis

Diagnosing Vulnerability: The 7 Points of Pipeline Failure



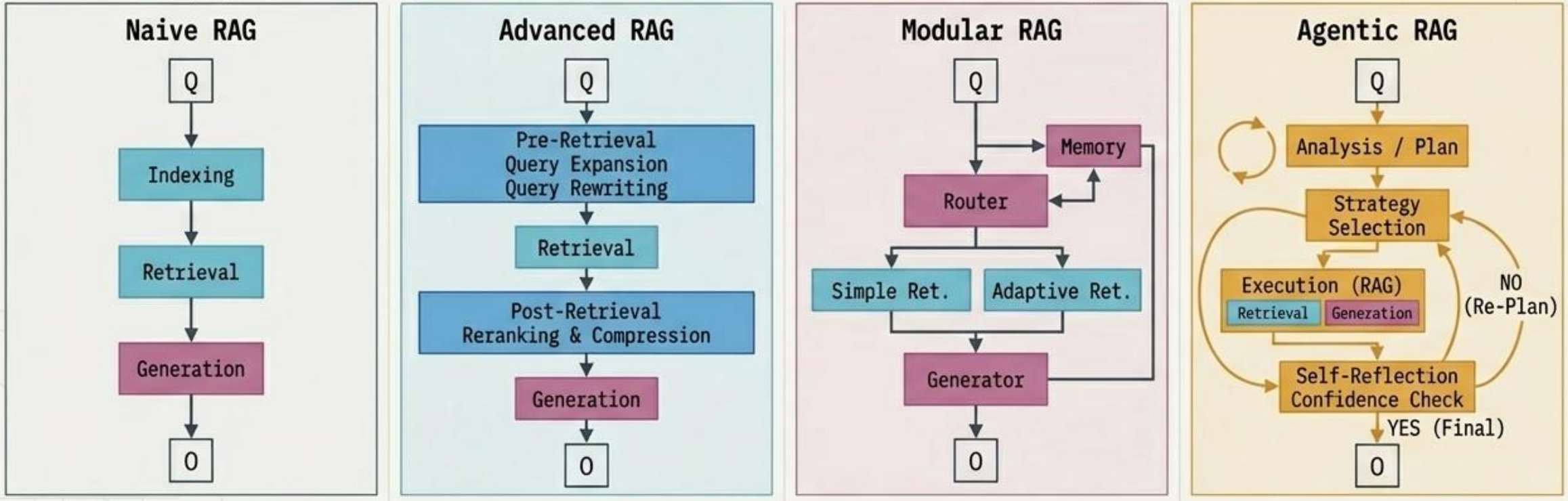
S1

RAG is not one architecture: it evolves through four stages

- Naive RAG: retrieve and generate
- Advanced RAG: optimize retrieval, context, and query handling
- Modular RAG: specialized components with fixed orchestration
- Agentic RAG: autonomous planning, retrieval, reflection, and revision

The RAG Maturity Model

Adding architectural complexity is a cost, not a goal. Modern RAG evolves from a static pipeline to dynamic loops based on strict reliability requirements.



Simple retrieve-then-generate pipeline suitable for proof-of-concept implementations

Enhanced pipeline with retrieval quality optimizations for production systems

Flexible architecture with specialized modules for complex reasoning tasks

Autonomous system with self-monitoring and adaptive behavior

Section 2 — Retrieval had to become semantic and scalable

If the system cannot find the right evidence under real phrasing and real scale, nothing downstream can save it

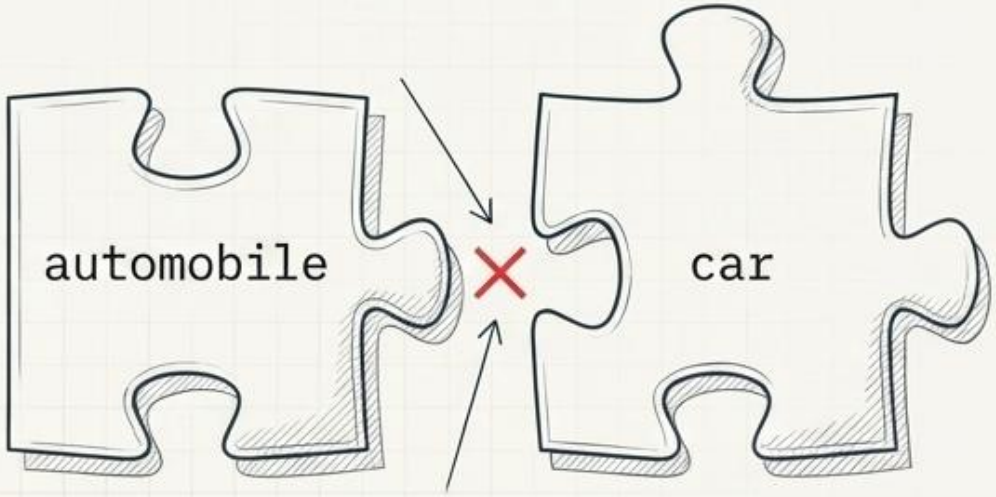
Word2Vec: from exact-match words to a map of meaning

S1

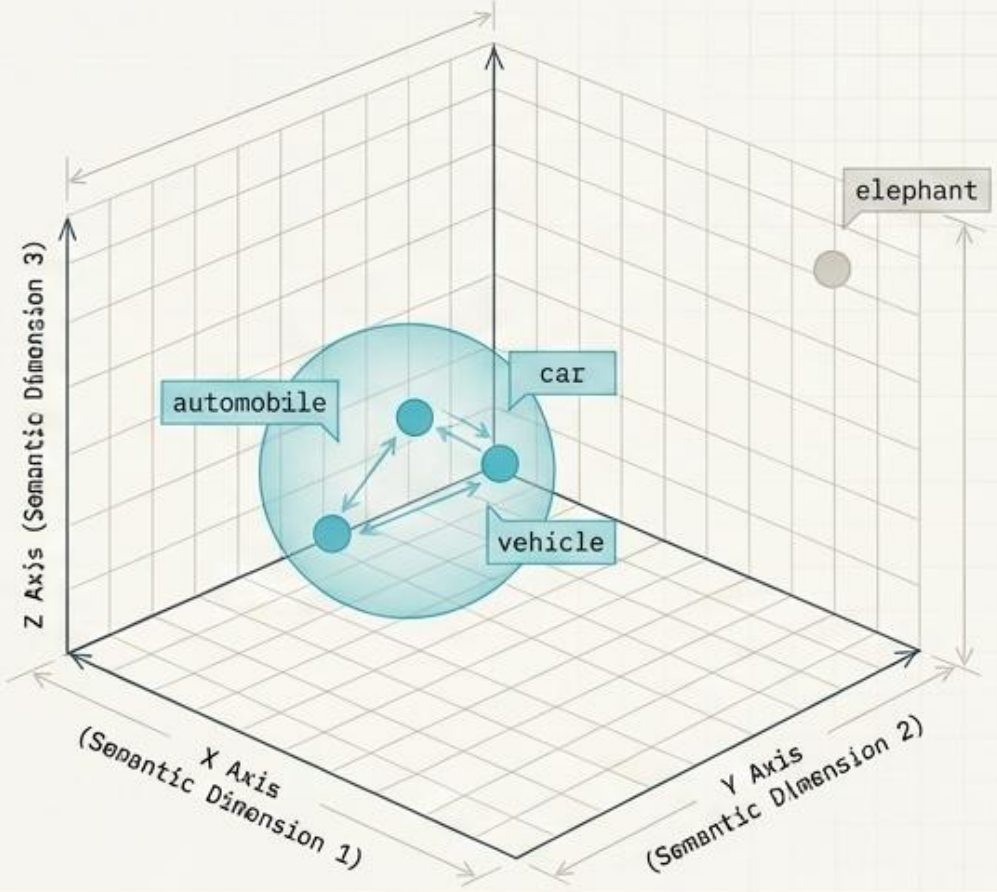
- Traditional lexical search treats 'automobile' and 'car' as unrelated symbols
- Word2Vec learns dense vectors from context, so related words end up near each other
- This is the first major fix for vocabulary mismatch in retrieval

The Foundations of Meaning: Word2Vec

Traditional keyword search (BM25) treats words as mathematically orthogonal entities; exact letters must match.



The Breakthrough: Word2Vec transitioned from sparse keywords to dense semantic space, proving that conceptual similarity equals geometric proximity.



Live demo: BM25 vs semantic retrieval on a toy corpus

S1

The lexical search finds documents sharing exact words such as 'network' and 'connection’

The semantic search also retrieves the WiFi ticket because the training corpus places 'wifi', 'internet', 'network', and 'connectivity' near one another."

FAISS + Sentence-BERT: semantics are useless if search is too slow

S1

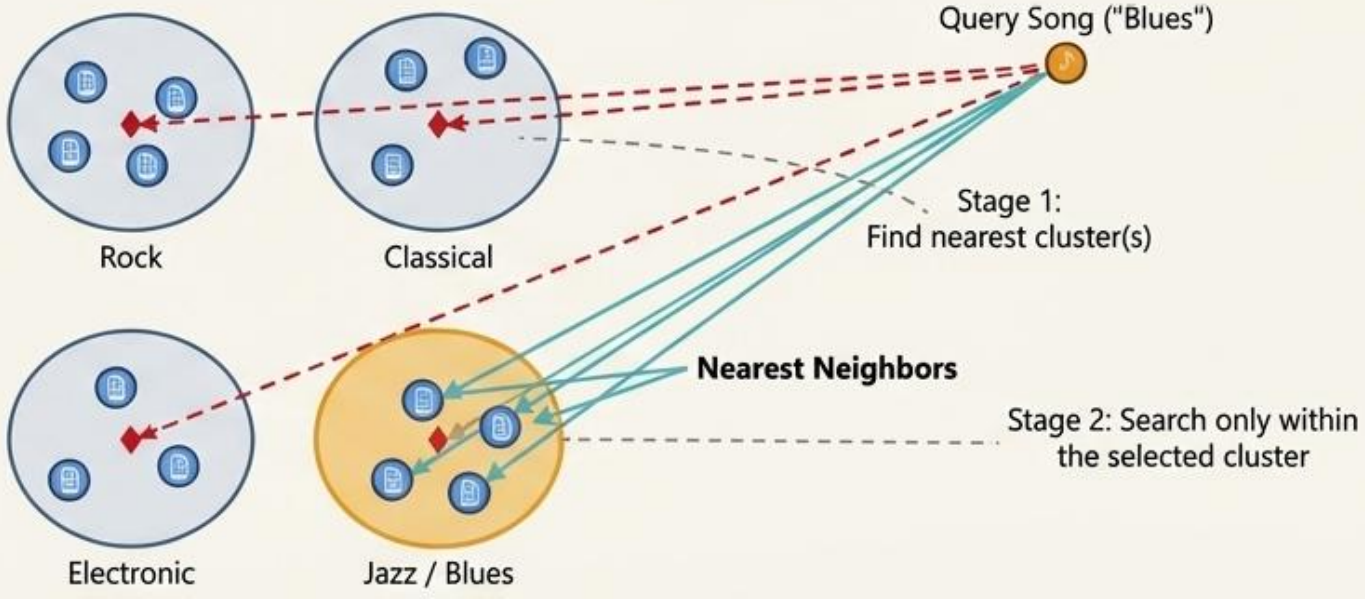
- FAISS makes approximate nearest-neighbor search practical at large scale
- Sentence-BERT turns BERT-style understanding into independently encodable sentence vectors
- Modern production retrieval often remains hybrid: sparse + dense

Billion-Scale Retrieval: The FAISS Indexing Trade-offs

Brute-force vector search requires O(n x d) operations. FAISS made semantic search computationally viable via approximate nearest neighbors (ANN).

Flat L2 (Exact Scan)	IVF (Inverted File)	HNSW (Graph Search)
Perfect accuracy, slow linear scaling.	Balance of speed and accuracy via clustering.	Ultra-fast, log-linear search but high RAM cost.
Best for: Critical medical/legal diagnosis (<100K vectors).	**Best for:** Enterprise document search (100K–10M vectors).	**Best for:** Real-time recommendations (>1M vectors).

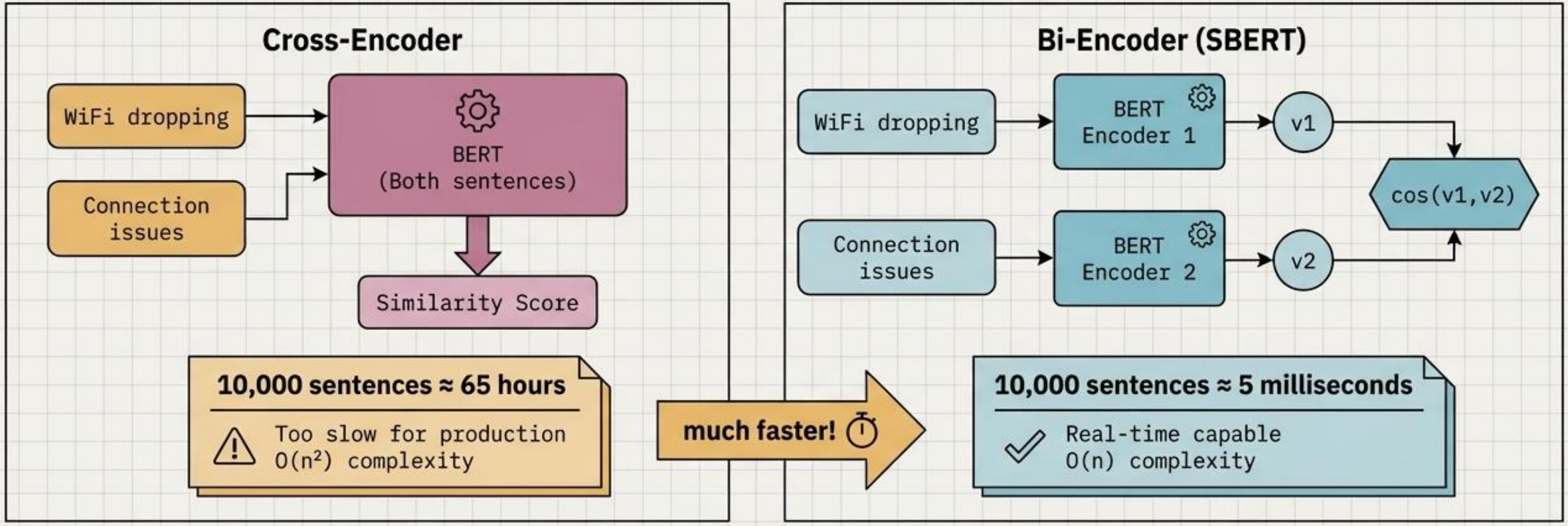
Clustering (IVF)



The Missing Link: Sentence-BERT & Bi-Encoders

The Bottleneck: BERT's original cross-encoder required both sentences to be processed together. Searching 10,000 documents took 65 hours.

The Fix: Sentence-BERT introduced siamese networks. By processing sentences independently, embeddings are pre-computed. Comparing them requires only a rapid dot a product, reducing search times to 5 milliseconds.



Live demo: ANN scale intuition

S1

Approximate search narrows the candidate pool before doing detailed scoring

That is the central idea behind scalable vector search systems such as FAISS."

Section 3 — Train retrieval and generation to cooperate

The next research leap was not 'retrieve more'; it was 'retrieve what actually helps the model think'

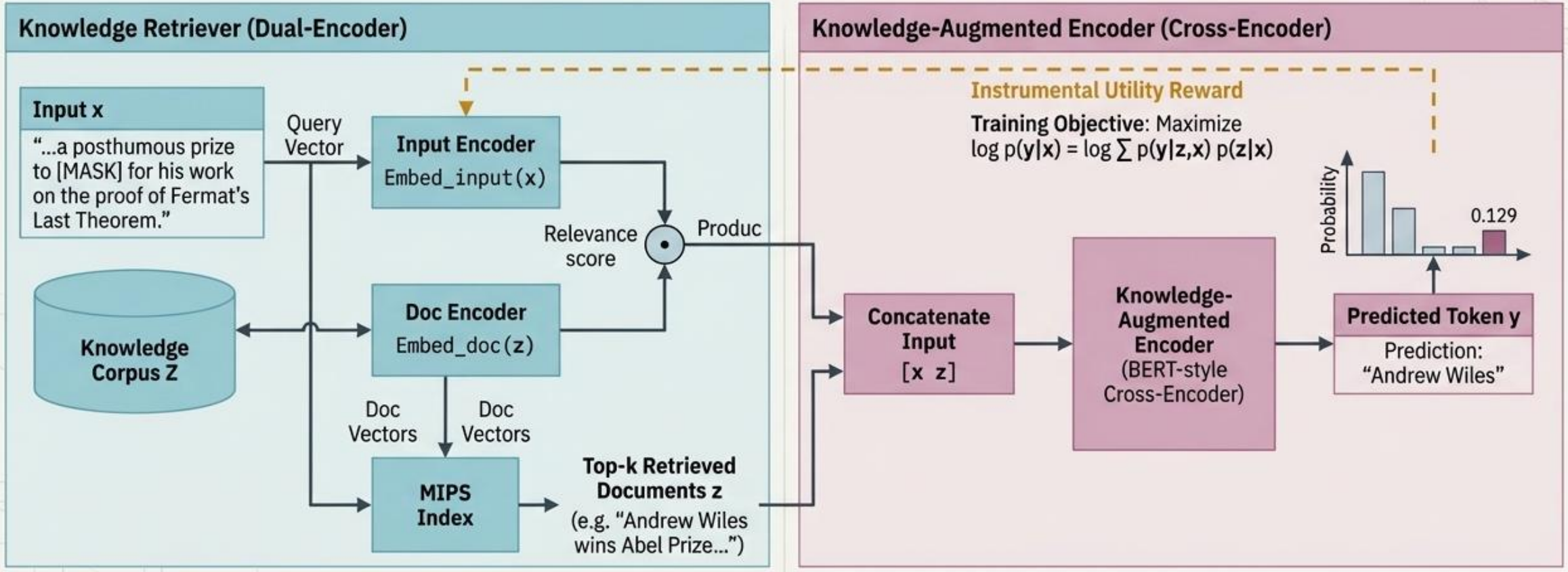
S2

REALM’s key move: retrieve documents with instrumental value

- Retriever and reader are trained end to end
- The reward signal is not just semantic similarity; it is whether retrieval improves prediction
- In plain language: the reader teaches the retriever what is useful

The Breakthrough: REALM and Instrumental Utility

The Paradigm Shift: REALM unfreezes the retriever, training it end-to-end with the language model. The retriever is only rewarded if the document it fetches helps the generator predict the masked word. Closeness in vector space is redefined by usefulness for answer prediction.



S2

Why REALM mattered even though most teams never shipped REALM directly

- It established explicit knowledge, modularity, interpretability, and performance-based retriever training
- It also exposed the operational pain: stale indexes, asynchronous refresh, and high training cost
- REALM is the conceptual ancestor; modular RAG is the deployable descendant

S2

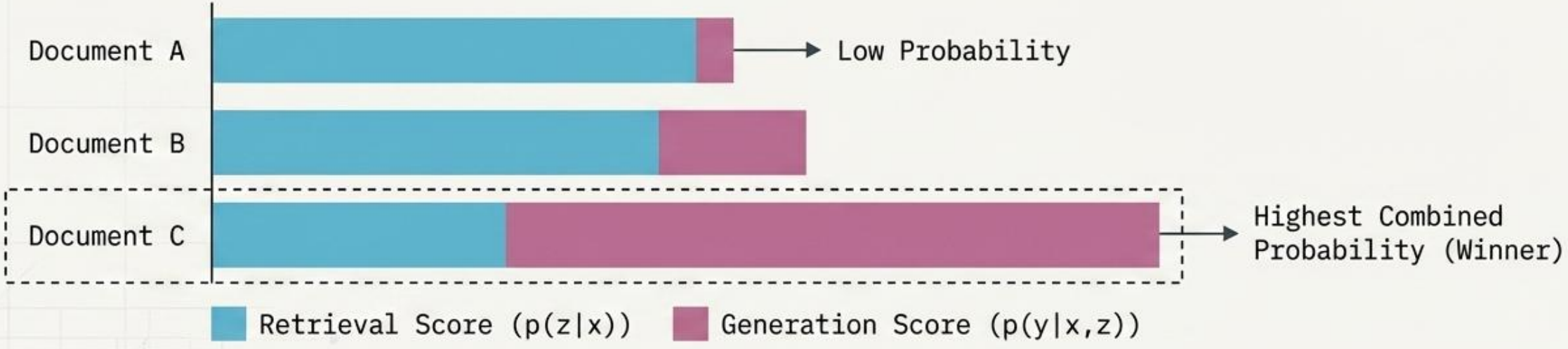
Canonical RAG: pretrained retriever + pretrained generator

- DPR retrieves; BART generates
- Knowledge lives in an external index, so the system can be updated without retraining the whole generator
- This modular design made RAG practical and economically accessible

Canonical RAG & The Power of Marginalization

The Modular Revolution: Lewis et al. (2020) proved that combining a frozen dense retriever (DPR) with a pretrained generator (BART) creates highly accurate systems. Instead of blindly trusting the top-1 retrieved document, RAG sums the probabilities across the top-K documents.

$$p(y|x) = \sum_{z \in Z} p(z|x) \cdot p(y|x, z)$$



Marginalization: why multiple documents beat top-1 retrieval

S2

- Canonical RAG models the answer as a weighted combination over top-k retrieved documents
- A lower-ranked document can still rescue the final answer if it provides stronger support
- This is the probabilistic fix for overtrusting the top result

S2

RAG-Sequence, RAG-Token, and why naive RAG won

- RAG-Sequence commits to one document per full answer
- RAG-Token can switch supporting documents token by token and is more expressive but slower
- Industry simplified further: retrieve top-k, stuff into the prompt, and use a strong LLM

Live demo: the minimal local RAG pipeline

S2

- Load and chunk documents
- Embed and index them
- Retrieve top-k chunks
- Stuff them into a prompt and answer

Section 4 — When one document is not enough

The next failure mode is not just missing evidence; it is failing to combine evidence that already exists

S3

FiD: independent encoding, fused decoding

- Each retrieved passage is encoded independently with the question
- Those encoded representations are fused only in the decoder
- This lets the system reason across many passages without one giant encoder input

S3

What FiD changes operationally

- The retriever can optimize more for recall than perfect top-1 precision
- FiD is especially good for FP5 Not Extracted and FP7 Incomplete
- Its main trade-off is slower decoding and heavier memory traffic

S3

Live demo: FiD-like fusion on the Odyssey example

- Encode each question + passage pair independently
- Reshape hidden states into one long decoder memory
- Compare a single-document answer with a fused multi-passage answer

S3

Atlas in practice: query-side tuning, freshness, compression

- A fixed retriever leaves performance on the table in few-shot settings
- Query-side tuning is often the cheapest useful adaptation
- Fresh indexes can flip time-sensitive answers; product quantization shrinks the index dramatically with little loss

Section 5 — Query-time retrieval gets smarter

What if retrieval fails not because the corpus is weak, but because the query is weak?

HyDE: imagine the answer-shaped document before search

S4

- Original query → hypothetical document → embed hypothetical document → retrieve real documents
- The hypothetical document is not evidence; it is a better search object
- HyDE is built for zero-shot retrieval when user wording and corpus wording do not match

Live Demo: HyDE Query Rewriting on the Upgrade/Session Example

S3

Start with the raw user question: “How can we stop customers from being kicked out during upgrades?”
Show baseline retrieval with the raw query and note that lexical search overweights words like customers and upgrades
Generate a hypothetical answer that introduces likely answer-language: session persistence, sticky sessions, Redis, rolling deployments
Retrieve again using the hypothetical document, then compare with query + hypothetical
Highlight how the top result shifts from a weakly related communications doc to the correct session-persistence runbook

Why HyDE works even when the pseudo-document is imperfect

S4

- The pseudo-document may contain invented details
- The contrastive encoder acts as a lossy bottleneck that preserves relevance pattern more than exact wording
- The final grounding still comes from retrieved real documents

S4

Live demo: direct retrieval vs HyDE on a policy corpus

- Direct retrieval underperforms on conversational wording
- A HyDE-style pseudo-document speaks the corpus's formal language
- The improvement is not magic; it is better query formation

S4

RAG-Fusion: do not trust one query if the problem has several angles

- Generate several related queries or sub-queries
- Retrieve for each one independently
- Merge the rankings with reciprocal rank fusion or a similar scheme
- Then rerank or filter before generation

Choosing among Atlas, HyDE, and RAG-Fusion

S4

- Atlas: train the retrieval-augmented learner better, especially for few-shot knowledge tasks
- HyDE: build a better search object when there are no relevance labels and the user asked in the wrong words
- RAG-Fusion: broaden coverage when one query formulation is too narrow

Section 6 — From better retrieval to better thinking

Later RAG systems add reflection, planning, hierarchy, and relational structure so the model can think with documents instead of merely fetching them

S5

Self-RAG + correction, planning, and reasoning

- Retrieval does not have to happen once at the start
- A stronger system can draft, critique, retrieve again, and revise
- Planning decomposes complex questions into smaller evidence needs
- Correction reduces confident overreach beyond the sources

S4

Live demo: multi-query retrieval plus self-reflective RAG on a project-failure corpus

- A single retrieval query tends to miss important angles in a broad “why did it fail?” question
- Query decomposition improves coverage by searching for timeline, ownership, technical, and stakeholder evidence separately
- The reflective retry is not magic; it simply detects missing support and issues a targeted follow-up retrieval

Live demo: flat retrieval vs GraphRAG on an incident corpus

S4

- Flat retrieval finds relevant fragments, but misses the causal chain
- GraphRAG follows connected evidence across the system graph
- The improvement is not magic; it is reasoning over relationships, not just chunks

S4

Live demo: flat retrieval vs RAPTOR on a mixed-topic corpus

- Flat retrieval searches isolated chunks and can miss the broader topic structure
- RAPTOR first retrieves a high-level summary branch, then drills down to supporting evidence
- The improvement is not magic; it is hierarchical query routing that adds theme-level context before detail retrieval

RAPTOR + GraphRAG: restructure the corpus before query time

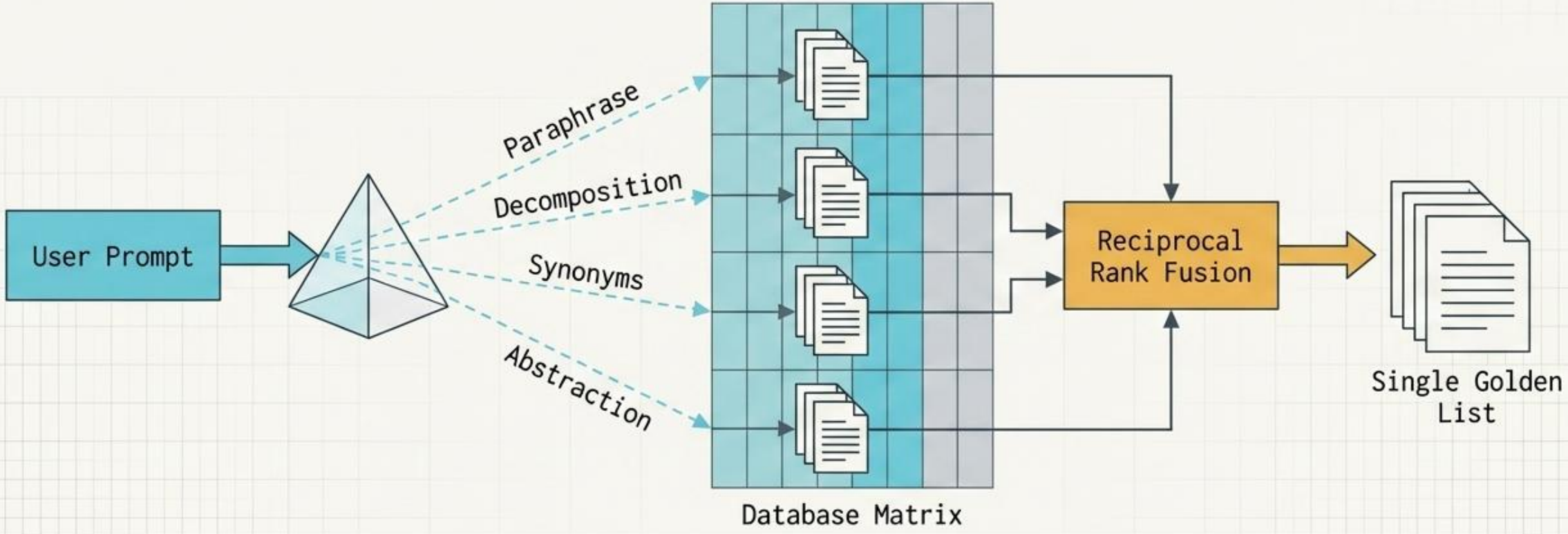
S5

- RAPTOR builds a hierarchy of summaries so retrieval can move between overview and detail
- GraphRAG retrieves neighborhoods of related entities, events, and claims instead of isolated chunks
- Both approaches improve retrieval by changing the knowledge structure, not just the prompt or the model

Fixing "Missed Top Rank": RAG-Fusion

The Vulnerability (FP2): Human questions are rarely complete specifications. A single query often locks onto the wrong vocabulary, missing critical documents.

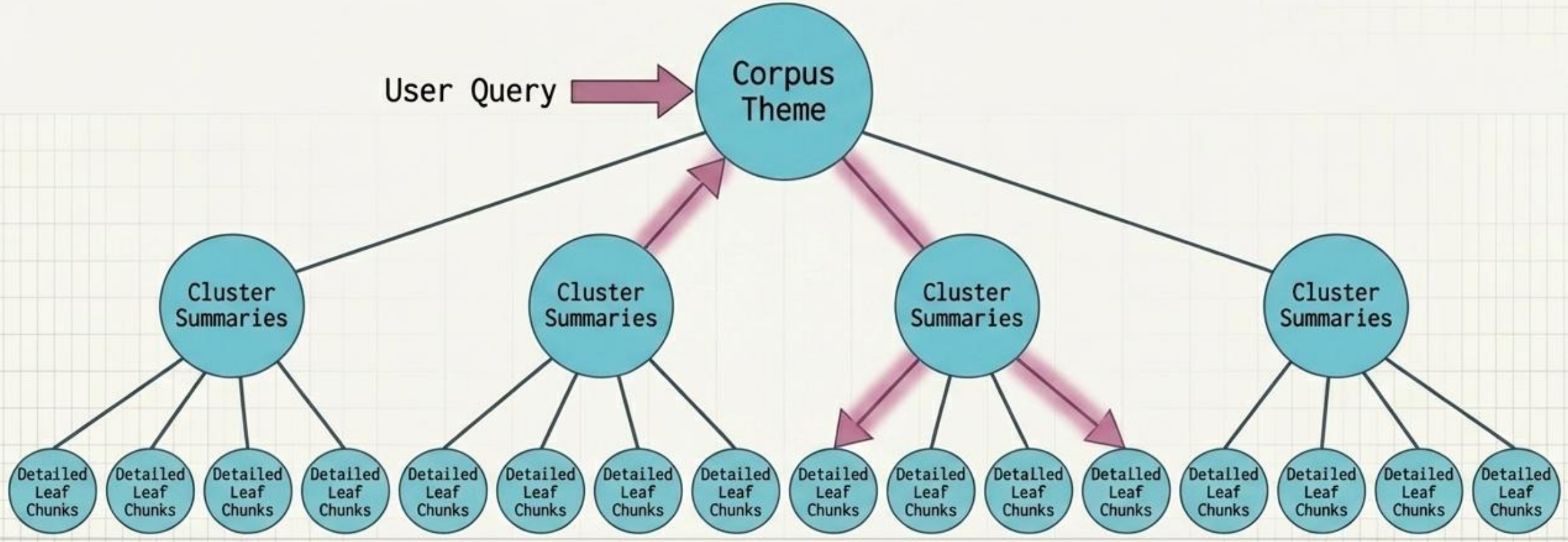
The Fix: RAG-Fusion forces the system to generate multiple semantic interpretations. It casts a wider net and relies on consistency via Reciprocal Rank Fusion to re-rank overlapping results.



Fixing "Incorrect Specificity": Tree-Organized Retrieval (RAPTOR)

The Vulnerability (FP6): Standard RAG treats the corpus as a flat bag of chunks. It fails on high-level thematic questions because it retrieves isolated fragments.

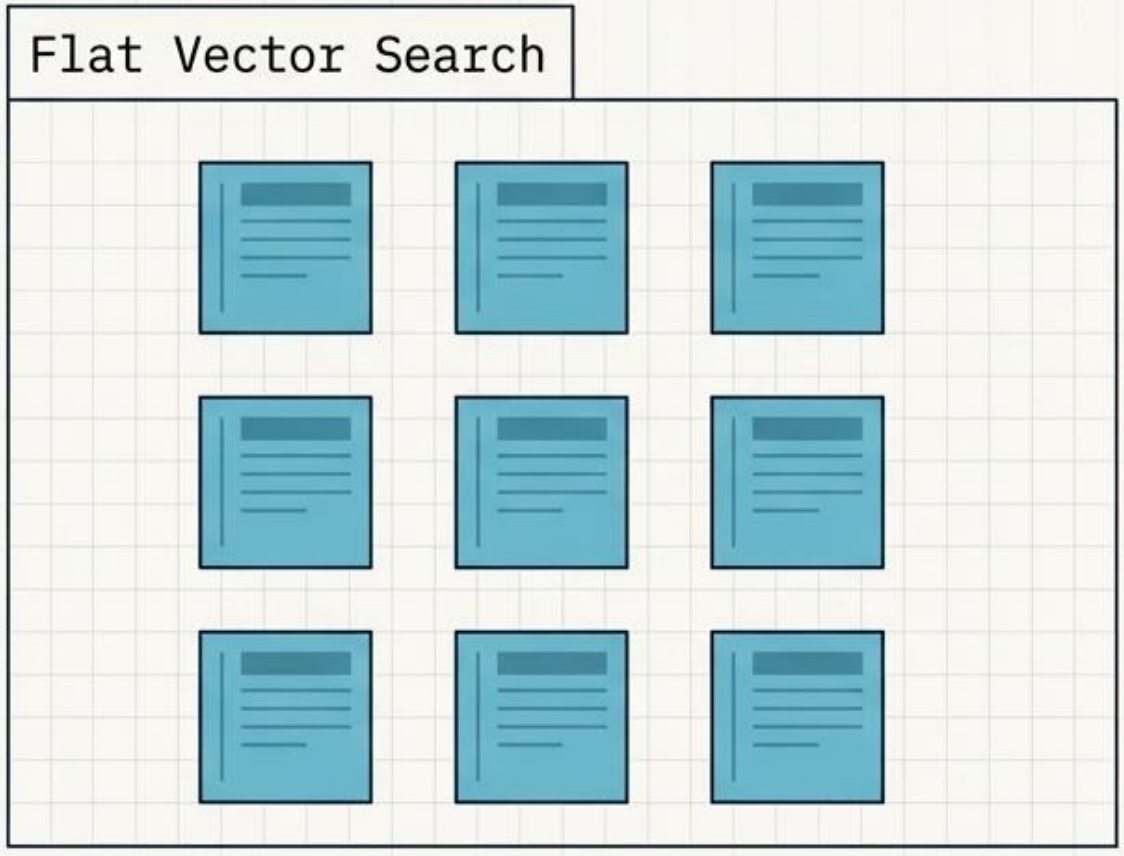
The Fix: RAPTOR clusters fine-grained segments and pre-computes abstractive summaries at multiple levels. Retrieval becomes a navigational scaffold.

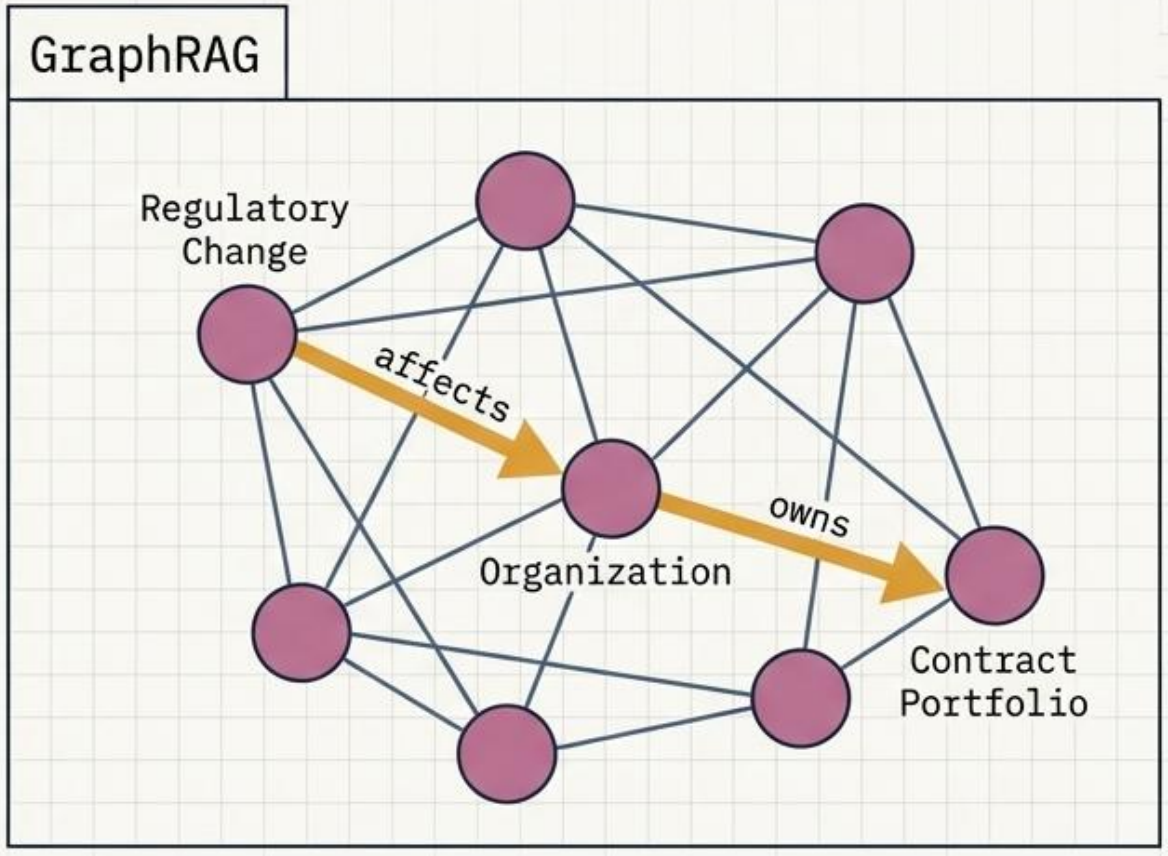


Fixing "Relational Blindspots": GraphRAG

The Vulnerability: Flat chunk retrieval finds text sharing vocabulary with a query, but fails at multi-hop questions requiring the synthesis of isolated facts.

The Fix: GraphRAG models the corpus as a structured knowledge network. It traces edges between entities to assemble answers that semantic similarity alone cannot see.

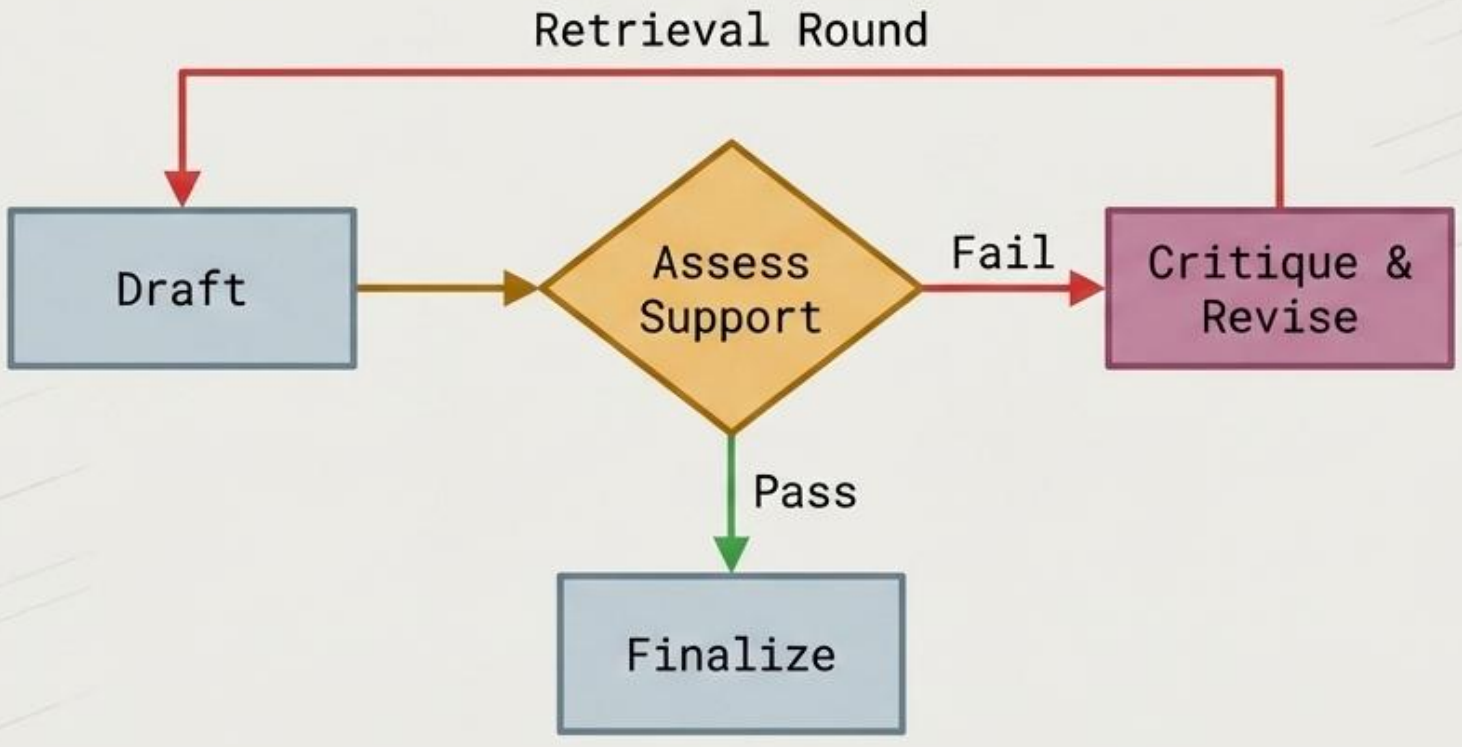




Fixing ‘Hallucination’: Self-RAG & Reflection

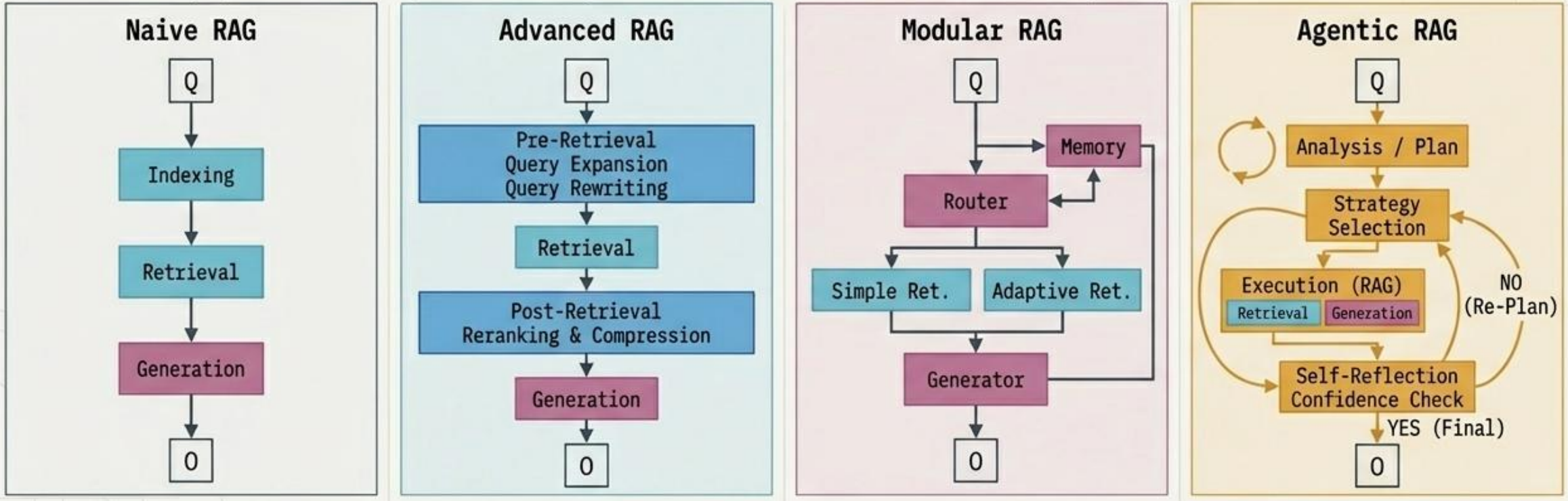
The Vulnerability (FP3): Language models are fluent before they are certain. They will confidently fill gaps with plausible but ungrounded language.

The Fix: Self-RAG treats retrieval, evaluation, and revision as an interconnected reflective loop. It introduces programmatic friction to prevent the first fluent answer from becoming the final answer.



The RAG Maturity Model

Adding architectural complexity is a cost, not a goal. Modern RAG evolves from a static pipeline to dynamic loops based on strict reliability requirements.



Simple retrieve-then-generate pipeline suitable for proof-of-concept implementations

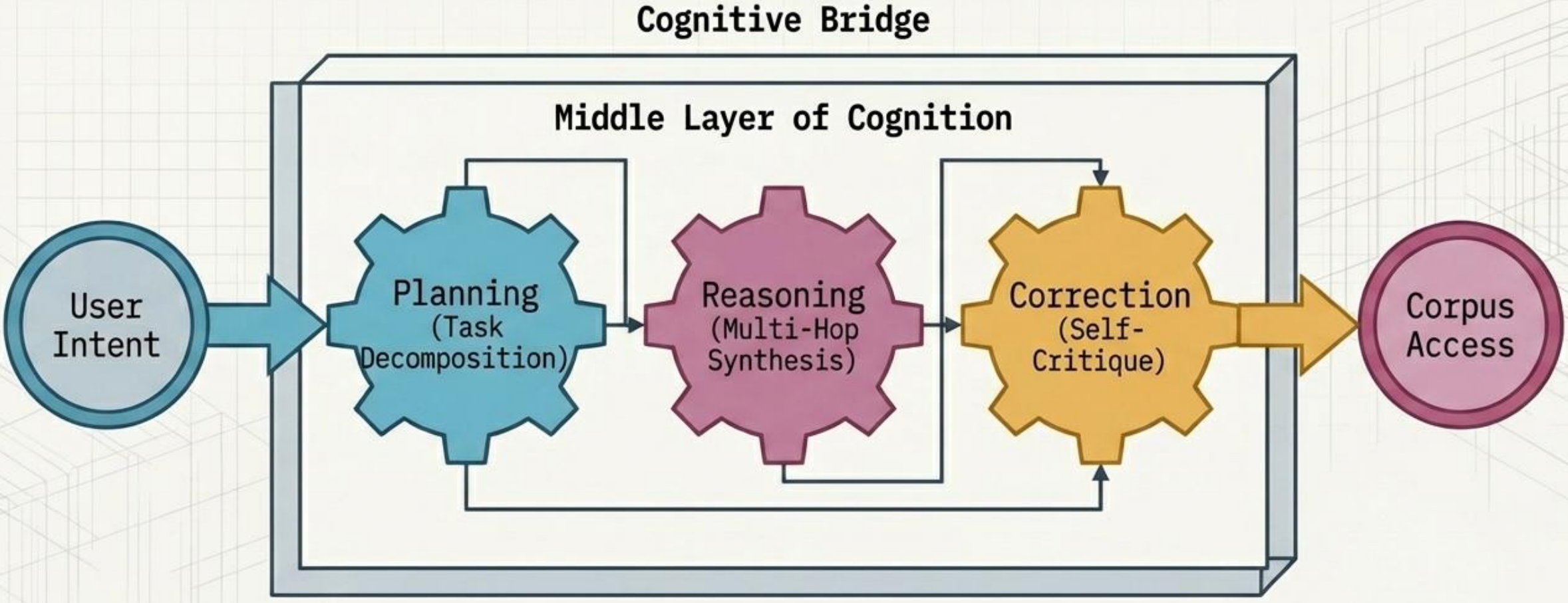
Enhanced pipeline with retrieval quality optimizations for production systems

Flexible architecture with specialized modules for complex reasoning tasks

Autonomous system with self-monitoring and adaptive behavior

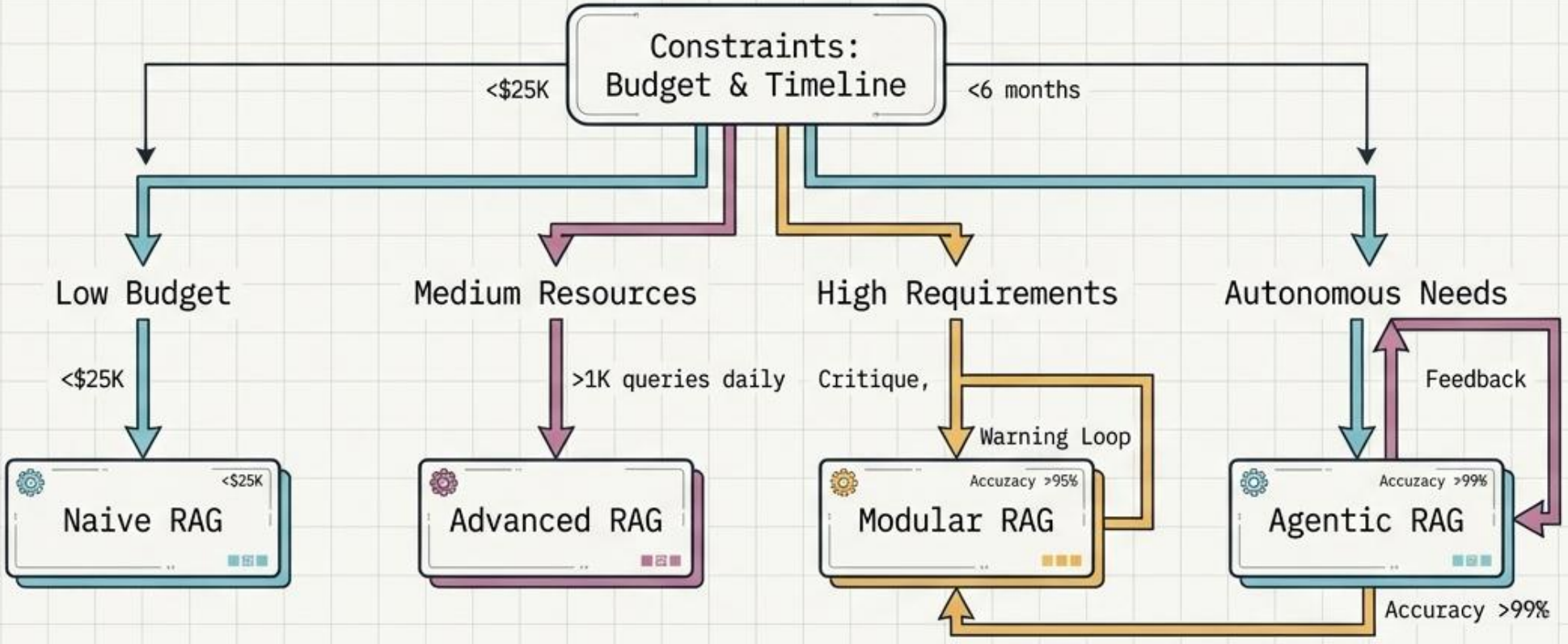
The Middle Layer of Cognition

The Synthesis: RAG is evolving from a simple search wrapper into a disciplined problem-solving engine. Systems don't just fetch documents; they actively think with documents.



Decision Synthesis: **Architecting for Your Context**

The Takeaway: Complexity is a cost. Apply the right architecture to your specific budget, scale, and reliability constraints.



Appendix

Speaker Notes: Chronological timeline of the key concepts, algorithms

Speaker Notes: Acronyms

Speaker Notes: Papers

Speaker Notes: Stuff

Speaker Notes: Sequence

Atlas: few-shot learning with retrieval augmentation

S3

- Atlas pairs a Contriever-style retriever with a FiD/T5-style generator
- Retrieval is used during training and fine-tuning, not bolted on afterward
- Result: a smaller retrieval-augmented system can behave like a much larger few-shot learner on knowledge tasks

Essential Bibliography of Retrieval-Augmented Generation and Transformers

S5

Foundational Models & Transformers

"Attention Is All You Need" (Vaswani et al., 2017): https://arxiv.org/abs/1706.03762
"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (Devlin et al., 2018): https://arxiv.org/abs/1810.04805
"Improving Language Understanding by Generative Pre-Training" (GPT-1): https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (T5, Raffel et al., 2019): https://arxiv.org/abs/1910.10683
"BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension" (Lewis et al., 2019): https://arxiv.org/abs/1910.13461